

Chapter 12. Scaling Workloads

There are several reasons why scaling a workload becomes necessary, particularly to maintain optimal performance under increasing demand. For example, an application may experience a surge in users as it gains popularity, or it may need to process larger volumes of data over time.

In Kubernetes, scaling a workload can be achieved in two primary ways: by increasing the resources allocated to each Pod (*vertical scaling*), or by adjusting the number of Pods running concurrently (*horizontal scaling*). Horizontal scaling is especially effective for handling fluctuating workloads, ensuring the application remains responsive and resilient under varying levels of demand, such as back pressures on CPU, memory, and I/O.

In this chapter, you'll learn how to manually scale the number of replicas as a reaction to increased application load. Furthermore, we'll explore the API primitive HorizontalPodAutoscaler (HPA), which allows you to automatically scale the managed set of Pods based on resource thresholds such as CPU and memory. We won't get into vertical scaling represented by the API primitive VerticalPodAutoscaler (VPA), as the exam won't cover it.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Configure workload autoscaling
-

Manual Scaling of Workload

Manual scaling of workloads requires specifying a fixed number of Pods to run. This number should be informed by real-world usage metrics gathered from production environments or estimated through load testing during development.

However, because application traffic can fluctuate unexpectedly—rising sharply or tapering off—you'll need to continuously monitor and adjust the number of Pods to match demand. Failing to do so can lead to over-

provisioning, which wastes resources, or under-provisioning, which can degrade performance and user experience.

Manually Scaling a Deployment

Scaling (up or down) the number of replicas controlled by a Deployment is a straightforward process. You can either manually edit the live object using the `edit deployment` command and change the value of the attribute `spec.replicas`, or you can use the imperative `scale deployment` command. In real-world production environments, you want to edit the Deployment YAML manifest, check it into version control, and apply the changes. The following command increases the number of replicas from four to six:

```
$ kubectl scale deployment app-cache --replicas=6
deployment.apps/app-cache scaled
```

You can observe the creation of replicas in real time using the `-w` command-line flag. You'll see a change of status for the newly created Pods turning from `ContainerCreating` to `Running`:

```
$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
app-cache-5d6748d8b9-6cc4j	1/1	ContainerCreating	0	11s
app-cache-5d6748d8b9-6rmlj	1/1	Running	0	28m
app-cache-5d6748d8b9-6z7g5	1/1	ContainerCreating	0	11s
app-cache-5d6748d8b9-96dzf	1/1	Running	0	28m
app-cache-5d6748d8b9-jkjsv	1/1	Running	0	28m
app-cache-5d6748d8b9-svrwx	1/1	Running	0	28m

Manually scaling the number of replicas takes a bit of guesswork. You will still have to monitor the load on your system to see if your number of replicas is sufficient to handle the incoming traffic.

Manually Scaling a StatefulSet

Another API primitive that can be scaled manually is the StatefulSet. StatefulSets are meant for managing stateful applications by a set of Pods (e.g., databases). Similar to a Deployment, the StatefulSet defines a Pod template; however, each of its replicas guarantees a unique and persistent identity. Similar to a Deployment, a StatefulSet uses a ReplicaSet to manage the replicas.

We are not going to look at StatefulSets in more detail, but you can read more about them in the [documentation](#). The reason I am discussing the

StatefulSet primitive here is that it can be manually scaled in a similar fashion as the Deployment.

Let's say we'd deal with the YAML definition for a StatefulSet that runs and exposes a Redis database, as illustrated in [Example 12-1](#).

Example 12-1. A YAML manifest for a StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  selector:
    matchLabels:
      app: redis
  replicas: 1
  serviceName: "redis"
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:6.2.5
          command: ["redis-server", "--appendonly", "yes"]
          ports:
            - containerPort: 6379
              name: web
          volumeMounts:
            - name: redis-vol
              mountPath: /data
      volumeClaimTemplates:
        - metadata:
            name: redis-vol
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 1Gi
```

After its creation, listing the StatefulSet shows the number of replicas in the `READY` column. As you can see in the following output, the number of replicas was set to `1`:

```
$ kubectl apply -f redis.yaml
service/redis created
statefulset.apps/redis created
$ kubectl get statefulset redis
```

NAME	READY	AGE
redis	1/1	2m10s

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
redis-0	1/1	Running	0	2m

The `scale` command we explored in the context of a Deployment works here as well. In the following command, we scale the number of replicas from one to three:

```
$ kubectl scale statefulset redis --replicas=3
```

```
statefulset.apps/redis scaled
```

```
$ kubectl get statefulset redis
```

NAME	READY	AGE
redis	3/3	3m43s

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
redis-0	1/1	Running	0	101m
redis-1	1/1	Running	0	97m
redis-2	1/1	Running	0	97m

It's important to mention that the process for scaling down a StatefulSet requires all replicas to be in a healthy state. Any long-term unresolved issues in Pods controlled by a StatefulSet can lead to a situation that can result in the application becoming unavailable to end users.

Autoscaling of Workload

Another way to scale a Deployment is with the help of a HorizontalPodAutoscaler (HPA). The HPA is an API primitive that defines rules for automatically scaling the number of replicas under certain conditions. Common scaling conditions include a target value, an average value, or an average utilization of a specific metric (e.g., for CPU and/or memory). Refer to the [MetricTarget API](#) for more information.

[Figure 12-1](#) shows an overview architecture diagram involving an HPA.

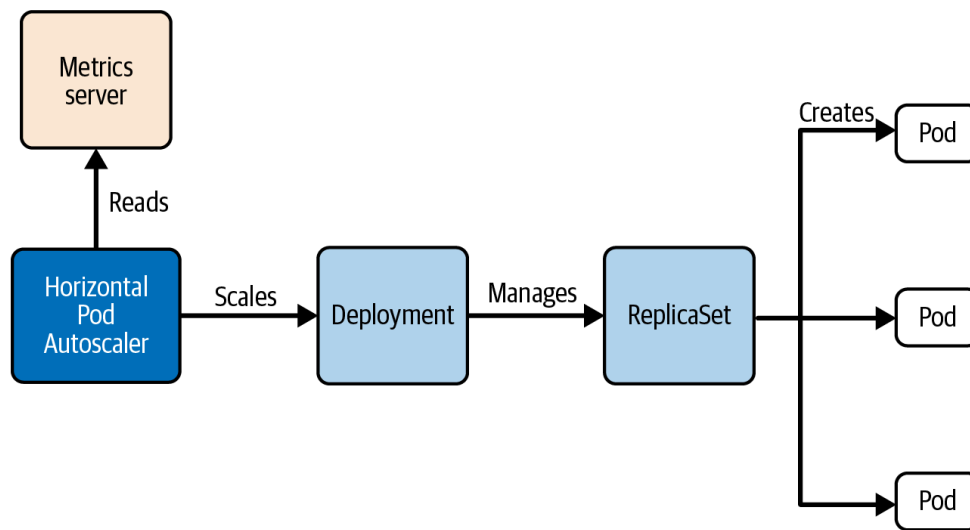


Figure 12-1. Autoscaling a Deployment

Prerequisites for Autoscaling

An HPA only works on scalable resources like the Deployment, ReplicaSet, and StatefulSet. It cannot scale standalone Pods. For an HPA to work, a couple of prerequisites need to be fulfilled:

- The [Metrics Server](#) needs to be installed. Without it, the HPA cannot retrieve the necessary metrics to evaluate Pod performance. Collecting metrics may take a couple of minutes initially after installing the component.
- The [container resource requests](#) need to be defined. For CPU-based autoscaling, your Pods must define `spec.containers[].resources.requests.cpu`. For memory-based autoscaling, `spec.containers[].resources.requests.memory` is required. These values provide the baseline for calculating utilization.
- The cluster must have sufficient CPU and memory resources to schedule new Pods.

Creating Horizontal Pod Autoscalers

Let's say you want to define average CPU utilization as the scaling condition. At runtime, the HPA checks the metrics collected by the Metrics Server to determine if the average maximum CPU or memory usage across all replicas of a Deployment is less than or greater than the defined threshold.

[Figure 12-2](#) shows the use of an HPA that will scale up the number of replicas if an average of 80% CPU utilization is reached across all available Pods controlled by the Deployment.

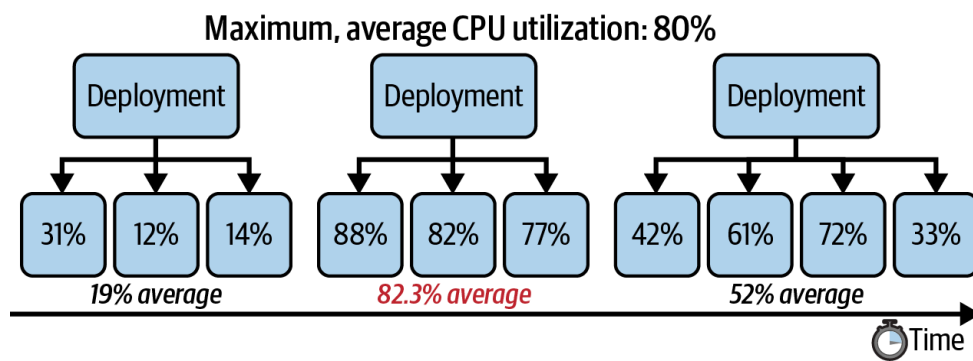


Figure 12-2. Autoscaling a Deployment horizontally

You can use the `autoscale deployment` command to create an HPA for an existing Deployment. The option `--cpu-percent` defines the average maximum CPU usage threshold. At the time of writing, the imperative command doesn't offer an option for defining the average maximum memory utilization threshold. The options `--min` and `--max` provide the minimum number of replicas to scale down to and the maximum number of replicas the HPA can create to handle the increased load, respectively:

```
$ kubectl autoscale deployment app-cache --cpu-percent=80 --min=3 --max=5
horizontalpodautoscaler.autoscaling/app-cache autoscaled
```

This command is a great shortcut for creating an HPA for a Deployment. The YAML manifest representation of the HPA object looks like

[Example 12-2.](#)

Example 12-2. A YAML manifest for an HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-cache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-cache
  minReplicas: 3
  maxReplicas: 5
  metrics:
  - resource:
      name: cpu
      target:
        averageUtilization: 80
        type: Utilization
    type: Resource
```

Listing Horizontal Pod Autoscalers

The short-form command for a Horizontal Pod Autoscaler is `hpa`. Listing all of the HPA objects transparently describes their current state: the CPU utilization and the number of replicas at this time:

```
$ kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
app-cache	Deployment/app-cache	<unknown>/80%	3	5	4

If the Pod template of the Deployment does not define CPU resource requirements or if the CPU metrics cannot be retrieved from the Metrics Server, the left-side value of the column `TARGETS` says `<unknown>`.

[Example 12-3](#) sets the resource requirements for the Pod template so that the HPA can work properly. You can learn more about defining resource requirements in [“Working with Resource Requirements”](#).

Example 12-3. Setting CPU resource requirements for Pod template

```
# ...
spec:
  # ...
  template:
    # ...
    spec:
      containers:
      - name: memcached
        # ...
        resources:
          requests:
            cpu: 250m
          limits:
            cpu: 500m
```

Once traffic hits the replicas, the current CPU usage is shown as a percentage. Here, the average maximum CPU utilization is 15%:

```
$ kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
app-cache	Deployment/app-cache	15%/80%	3	5	4

Rendering Horizontal Pod Autoscaler Details

The event log of an HPA can provide additional insight into the rescaling activities. Rendering the HPA details can be a great tool for overseeing

when the number of replicas was scaled up or down, as well as their scaling conditions:

```
$ kubectl describe hpa app-cache
```

```
Name: app-cache
Namespace: default
Labels: <none>
Annotations: <none>
CreationTimestamp: Sun, 15 Aug 2021 \
15:54:11 -0600
Reference: Deployment/app-cache
Metrics: ( current / target
  resource cpu on pods (as a percentage of request): 0% (1m) / 80%
Min replicas: 3
Max replicas: 5
Deployment pods: 3 current / 3 desired
Conditions:
  Type           Status  Reason                        Message
  ----           -
  AbleToScale    True    ReadyForNewScale             recommended size matches current
  ScalingActive  True    ValidMetricFound             the HPA was able to successfully
  calculate a replica count from cpu resource utilization (percentage of request)
  ScalingLimited True    TooFewReplicas               the desired replica count is less
  than the minimum replica count
Events:
  Type           Reason             Age   From                  Message
  ----           -
  Normal        SuccessfulRescale   13m   horizontal-pod-autoscaler  New size: 3;
  reason: All metrics below target
```

Defining Multiple Scaling Metrics

You can define more than a single resource type as a scaling metric. As you can see in [Example 12-4](#), we are inspecting CPU and memory utilization to determine if the replicas of a Deployment need to be scaled up or down.

Example 12-4. A YAML manifest for a HPA with multiple metrics

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-cache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-cache
  minReplicas: 3
  maxReplicas: 5
```



```

metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 80
- type: Resource
  resource:
    name: memory
    target:
      type: AverageValue
      averageValue: 500Mi

```

To ensure that the HPA determines the currently used resources, we'll set the memory resource requirements for the Pod template as well, as shown in [Example 12-5](#).

Example 12-5. Setting memory resource requirements for Pod template

```

...
spec:
  ...
  template:
    ...
    spec:
      containers:
      - name: memcached
        ...
        resources:
          requests:
            cpu: 250m
            memory: 100Mi
          limits:
            cpu: 500m
            memory: 500Mi

```

Listing the HPA renders both metrics in the `TARGETS` column, as in the output of the `get` command shown here:

```

$ kubectl get hpa

```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
app-cache	Deployment/app-cache	1994752/500Mi, 0%/80%	3	5
3	2m14s			

Summary

Scaling workloads manually requires deep insight into the requirements and the load of an application. A Horizontal Pod Autoscaler can automatically scale the number of replicas based on CPU and memory thresholds observed at runtime.

Exam Essentials

Understand the difference between manual and automatic scaling

Kubernetes workloads can be scaled either manually or automatically. In real-world scenarios, autoscaling is the preferred approach, as it allows the number of replicas to adjust dynamically based on actual resource consumption relative to defined thresholds. This ensures optimal performance and resource efficiency without the need for constant manual oversight.

Identify if autoscaling prerequisites are fulfilled

For the HPA to function correctly, it's essential to have the Metrics Server installed and to define resource requests for your containers. Without these in place, the HPA won't have the necessary data to make informed scaling decisions, rendering it ineffective.

Know how to instantiate and configure a Horizontal Pod Autoscaler

An HPA YAML manifest is built around three essential components: the scaling target (the named resource the HPA will manage, e.g., a Deployment or a ReplicaSet), the minimum and maximum number of replicas the HPA can scale between, and the scaling rules—the conditions that trigger scaling, typically based on resource usage thresholds like CPU or memory utilization.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Define a Deployment named `hello-world` with three replicas in a YAML file named `hello-world-deployment.yaml`. The Pod template of the Deployment should use the container image `bmuschko/nodejs-hello-world:1.0.0`. Create the object from the YAML file.
Scale the number of replicas to eight by modifying the YAML file. Apply the changes and verify that all eight replicas are running.

2. Create a Deployment named `nginx` with one replica. The Pod template of the Deployment should use container image `nginx:1.23.4`; set the CPU resource request to 0.5 and the memory resource request/limit to 500 Mi.
Create a HorizontalPodAutoscaler for the Deployment named `nginx-hpa` that scales to a minimum of three and a maximum of eight replicas. Scaling should happen based on an average CPU utilization of 75% and an average memory utilization of 60%.
Inspect the HorizontalPodAutoscaler object and identify the currently-utilized resources. How many replicas do you expect to exist?